DISSERTATION REPORT

ON

"WEB BASED ADMISSION ENQUIRY SYSTEM"

Submitted in partial fulfilment of the requirements for the award of the degree of

BACHELOR OF COMPUTER APPLICATIONS (BCA)

Semester VI

Submitted By

Name of Student: Sumit Sarswat

Roll No.: 101301

University Enrolment No. : _____

Under the Guidance of

Name of Supervisor: Mr. Umesh Kumar Tanwar

Department of Computer Applications

B.J.S. Rampuria Jain College, Bikaner

Academic Session: 2026-27

DISSERTATION REPORT

"WEB BASED ADMISSION ENQUIRY SYSTEM"

A Dissertation Report submitted to

B.J.S. RAMPURIA JAIN COLLEGE, BIKANER

in partial fulfilment of the requirements for the award of

BACHELOR OF COMPUTER APPLICATIONS (BCA)

Semester VI

Submitted By

Name: Sumit Sarswat

Roll No.: 101301

Enrolment No. : _____

Under the Guidance of

Name of Supervisor: Mr. Umesh Kumar Tanwar

Department of Computer Applications

B.J.S. Rampuria Jain College, Bikaner

Academic Session: 2026-27

CERTIFICATE

This is to certify that Mr. Sumit Sarswat, Roll No. 101301, Enrolment No._____, student of BCA Semester VI, B.J.S. Rampuria Jain College, Bikaner, has successfully completed the dissertation project entitled "Web Based Admission Enquiry System" under my guidance and supervision during the Academic Session 2026-27.

The work presented in this dissertation is based on the student's study, analysis, development, and implementation carried out as part of the academic requirements of the BCA Semester VI programme.

To the best of my knowledge, the work submitted by the student is suitable for evaluation as a Semester VI Dissertation Project.

Date: .....

Place: Bikaner

Signature of Project Supervisor

Name: Mr. Umesh Kumar Tanwar

DECLARATION

I, Sumit Sarswat, student of BCA Semester VI, Roll No. 101301, hereby declare that the dissertation project entitled "Web Based Admission Enquiry System" submitted by me to B.J.S. Rampuria Jain College, Bikaner, during the Academic Session 2026-27, is my original academic work carried out under the guidance of Mr. Umesh Kumar Tanwar.

I further declare that this project has been prepared for the purpose of academic evaluation and has not been submitted previously, either fully or partially, for the award of any other degree, diploma, or certificate at this or any other educational institution.

Wherever information, concepts, code, images, datasets, or other material from external sources have been used, appropriate acknowledgement and references have been provided in the bibliography and reference sections of this report.

I understand that plagiarism, fabrication of results, submission of copied work, or misrepresentation of another person's work may lead to rejection of the project and/or disciplinary action as per the institutional rules of B.J.S. Rampuria Jain College and the affiliated university.

Date: .....

Place: Bikaner

Signature of Student: .....

Name: Sumit Sarswat

ACKNOWLEDGEMENT

I would like to express my sincere gratitude to the Principal of B.J.S. Rampuria Jain College, Bikaner, for providing me with the opportunity, institutional resources, and necessary academic support to undertake and successfully complete this dissertation project.

I am thankful to my project supervisor, Mr. Umesh Kumar Tanwar, for his valuable guidance, suggestions, encouragement, and continuous support during the preparation, coding, testing, and documentation of this project.

I also express my gratitude to the faculty members of the Department of Computer Applications for providing the academic knowledge and technical foundation required for undertaking this work.

I am thankful to my classmates, friends, and family members for their encouragement, patience, and support during the completion of the project.

Finally, I acknowledge the books, websites, software tools, libraries, and online documentation used during the development and study of the project.

Name of Student: Sumit Sarswat

Roll No.: 101301

ABSTRACT

The project entitled **"Web Based Admission Enquiry System"** focuses on **making the admission enquiry process easier and more accessible for students and college staff through an online portal**.

The primary objective of the project is to **allow students to read college information, interact with a chatbot for common questions, and securely submit enquiry forms without visiting the college in person**. The project has been developed using **Python with the Flask framework, Jinja2, HTML, CSS, and JavaScript** and makes use of **a MySQL database for storing enquiry and administrator data**.

The proposed system/project provides facilities such as **an online enquiry submission form**, **a browser-based interactive chatbot**, and **a secure admin dashboard to view and manage submitted enquiries from a central place**.

The methodology adopted for the project includes study of the existing system, identification of requirements, system design, implementation, testing and evaluation of the developed solution.

The developed project was tested using appropriate test cases to verify its major functionalities. The results indicate that the proposed system can be used for **reducing manual enquiry work and efficiently managing admission information through a simple online interface**.

The project provided practical exposure to concepts related to **web development, database connectivity, authentication, client-side scripting, and software testing** and helped in understanding the process of converting theoretical computer application concepts into a practical solution.

TABLE OF CONTENTS

1. Certificate ii
2. Declaration iii
3. Acknowledgement iv
4. Abstract v
5. List of Figures viii
6. List of Tables ix
7. List of Abbreviations x
8. CHAPTER 1 - INTRODUCTION 1
8.1 Background of the Project 1
8.2 Introduction to the Problem 1
8.3 Problem Statement 2
8.4 Need for the Project 2
8.5 Objectives of the Project 2
8.6 Scope of the Project 3
8.7 Significance of the Project 4
8.8 Organization of the Dissertation 4
9. CHAPTER 2 - LITERATURE REVIEW / BACKGROUND STUDY 6
9.1 Introduction to the Subject Area 6
9.2 Existing Systems and Related Work 6
9.2.1 Traditional Manual System 6
9.2.2 Online Forms and Spreadsheets 7
9.2.3 Large Educational ERP Systems 7
9.2.4 Third-Party Education Portals 7
9.3 Review of Similar Applications and Technologies 7
9.4 Limitations of Existing Systems 8
9.5 How the Proposed System Differs 8
9.6 Summary of the Review 8
10. CHAPTER 3 - SYSTEM ANALYSIS AND REQUIREMENTS 9
10.1 Existing System 9
10.2 Proposed System 9
10.3 Functional Requirements 9
10.4 Non-Functional Requirements 10
10.5 User Requirements 10
10.6 Hardware Requirements 10
10.7 Software Requirements 11
10.8 Feasibility Study 11
10.9 Requirement Summary 11
10.10 Security and Data Integrity Requirements 12
10.11 Analysis Summary 12
11. CHAPTER 4 - SYSTEM DESIGN AND METHODOLOGY 13
11.1 Development Methodology 13
11.2 System Architecture 13
11.3 System Flow 13
11.4 Database Design 14
11.4.1 Enquiries Table 15
11.4.2 Admin Table 15

TABLE OF CONTENTS

11.5 Database Relationships 15
11.6 Security Design 15
11.7 Design Summary 16
11.8 Input and Output Design 16
11.9 Route and Module Summary 16
11.10 Validation and Error Handling Design 17
11.11 Design Summary 17
11.12 User Roles and Permission Design 17
11.13 Design Decisions 17
12. CHAPTER 5 - IMPLEMENTATION 19
12.1 Development Environment 19
12.2 Project Directory Structure 19
12.3 Main Modules of the System 19
12.4 Backend Implementation 20
12.4.1 Database Connection 20
12.4.2 Enquiry Submission 20
12.4.3 Admin Dashboard Access 21
12.5 Frontend Implementation 22
12.5.1 Admin Dashboard Template 22
12.6 Chatbot Integration 22
12.7 Chatbot Keyword Rules 24
12.8 Implementation Screenshots 24
12.9 Implementation Summary 27
12.10 Frontend Page Summary 27
12.11 Database Operations and Error Handling 28
12.12 Administrative Workflow 28
12.13 Implementation Notes 28
12.14 Maintenance and Deployment Notes 28
12.15 Implementation Summary 29
13. CHAPTER 6 - TESTING AND RESULTS 30
13.1 Testing Methodology 30
13.2 Test Cases 30
13.3 Test Results 30
13.4 Security Testing 31
13.5 Discussion of Results 31
13.6 Requirement-to-Test Traceability 31
13.7 Overall Test Summary 32
13.8 Limitations of Testing 32
14. CHAPTER 7 - CONCLUSION AND FUTURE SCOPE 33
14.1 Conclusion 33
14.2 Major Findings 33
14.3 Limitations 34
14.4 Future Scope 34
14.5 Final Summary 35
15. REFERENCES / BIBLIOGRAPHY 35

LIST OF FIGURES

Figure No.	Caption	Page No.
Figure 4.1	Overall System Architecture	14
Figure 4.2	Enquiry Submission Flow	15
Figure 5.1	Home Page	24
Figure 5.2	Admission Enquiry Form - Personal Information	25
Figure 5.3	Admission Enquiry Form - Academic and Query Details	26
Figure 5.4	Admin Login Page	26
Figure 5.5	Admin Dashboard	27
Figure 5.6	Chatbot Interaction	27

LIST OF TABLES

Table No.	Title	Page No.
Table 3.1	Functional Requirements	10
Table 3.2	Non-Functional Requirements	10
Table 3.3	Hardware Requirements	11
Table 3.4	Software Requirements	11
Table 3.5	Feasibility Summary	11
Table 3.6	Security and Data Integrity Requirements	12
Table 4.1	Development Phases	13
Table 4.2	Enquiries Table Data Dictionary	15
Table 4.3	Admin Table Data Dictionary	15
Table 4.4	Input and Output Design	16
Table 4.5	Main Route and Module Summary	17
Table 4.6	User Roles and Permission Design	17
Table 5.1	Main Project Modules	20
Table 5.2	Chatbot Keyword Rules	24
Table 5.3	Frontend Page Summary	28
Table 6.1	Detailed Test Cases	30
Table 6.2	Requirement-to-Test Traceability	32
Table 6.3	Overall Test Summary	32

LIST OF ABBREVIATIONS

Abbreviation	Meaning
BCA	Bachelor of Computer Applications
DB	Database
DFD	Data Flow Diagram
ERD	Entity-Relationship Diagram
ERP	Enterprise Resource Planning
FAQ	Frequently Asked Questions
HTML	HyperText Markup Language
CSS	Cascading Style Sheets
JS	JavaScript
MVC	Model-View-Controller
NLP	Natural Language Processing
POST	HTTP request method used to send data
SQL	Structured Query Language
UI	User Interface
URL	Uniform Resource Locator
QA	Quality Assurance
API	Application Programming Interface
AJAX	Asynchronous JavaScript and XML

CHAPTER 1 - INTRODUCTION

1.1 Background of the Project

Educational institutions are constantly evolving to adapt to the technological advancements of the 21st century. The foundation of any academic institute relies heavily on its admission process, which serves as the primary gateway for prospective students. Over the years, the influx of students seeking higher education has grown exponentially. Consequently, colleges are faced with the monumental task of managing inquiries, disseminating accurate course information, and guiding students through admission protocols.

In a traditional setup, this process involves extensive paperwork, physical helpdesks, and telephonic conversations. However, as the digital divide narrows, students now expect seamless, on-demand access to information. A Web-Based College Admission Enquiry System addresses this paradigm shift by leveraging web technologies to create a virtual helpdesk. This project is built using modern web development frameworks—specifically Python with Flask for the backend and MySQL for robust data management—to ensure high availability, scalability, and security. By transitioning from a physical inquiry model to a digital one, the college can significantly optimize its administrative resources while providing a highly interactive and user-friendly experience to applicants.

1.2 Introduction to the Problem

Despite the ubiquitous nature of the internet, many educational establishments still utilize fragmented or legacy systems to handle admission inquiries. Prospective students often find themselves navigating through poorly structured websites, forcing them to resort to phone calls or emails for basic information regarding course availability, faculty details, and fee structures.

The core problem lies in the inefficiency of manual information retrieval and dissemination. When admission counselors are burdened with answering repetitive questions, their productivity diminishes, leading to bottlenecks during peak admission cycles. Furthermore, prospective students facing delayed responses or lack of 24/7 support may lose interest, affecting the institution's enrollment rates. There is also the critical issue of data mismanagement; inquiries recorded in physical ledgers or disjointed

spreadsheets are difficult to track, follow up on, and analyze for future strategic planning. This project introduces a systematic, automated approach to mitigate these operational inefficiencies.

1.3 Problem Statement

The objective is to design, develop, and deploy a comprehensive 'Web-Based College Admission Enquiry System'. This system aims to provide a centralized, interactive, and responsive web portal for prospective students to seamlessly access course details, faculty profiles, and frequently asked questions. Simultaneously, it intends to deliver a secure, authenticated administrative dashboard for college staff to monitor, manage, and respond to incoming admission inquiries in real-time, thereby eliminating the dependencies on manual data entry and disjointed communication channels.

1.4 Need for the Project

The implementation of this project is driven by several pressing needs from both the user's (student's) and the administrator's (college's) perspectives:

Accessibility and Convenience: Today's students are digital natives who prefer accessing information at their convenience, regardless of operating hours. A dedicated portal fulfills this need by providing 24/7 access to crucial academic information.

Real-Time Query Resolution: Integrating an interactive, rule-based chatbot directly into the portal caters to the need for instant support. It can independently handle a large volume of standard inquiries, reducing the wait time for students to zero.

Centralized Data Management: The administration requires a unified database to store applicant details securely. Utilizing a relational database like MySQL ensures that inquiry records are consistent, easily retrievable, and safe from unauthorized access.

Resource Optimization: By automating the initial phases of the admission inquiry process, the college can reallocate its human resources to more critical tasks, such as personalized counseling for shortlisted candidates, rather than answering repetitive administrative questions.

1.5 Objectives of the Project

The project is guided by the following specific and measurable objectives:

1. To develop a dynamic, responsive web interface using HTML, CSS, and Vanilla JavaScript that adapts seamlessly across various devices (desktops, tablets, and smartphones).
2. To engineer a robust backend server using the Python Flask micro-framework, facilitating efficient routing, session management, and server-side logic.
3. To design a normalized relational database architecture using MySQL to accurately model courses, subjects, faculty profiles, FAQs, and student inquiries.
4. To implement a secure RESTful-like mechanism for handling form submissions asynchronously using AJAX, ensuring a smooth user experience without unnecessary page reloads.
5. To create a secure Administrative Control Panel protected by modern authentication practices, including password hashing (Werkzeug security) and SQL Injection prevention (via PDO/PyMySQL prepared statements), allowing staff to efficiently manage the inquiry lifecycle.

1.6 Scope of the Project

The scope of the Web-Based College Admission Enquiry System is clearly defined to ensure a focused and functional prototype within the stipulated timeframe.

Functional Boundaries:
- Public Portal: Showcasing dynamic data for college courses, detailed semester-wise subject breakdowns, department-wise faculty directories, and an FAQ section.
- Chatbot Module: A client-side conversational interface designed to parse user inputs and return predefined, context-aware responses based on keyword matching.
- Inquiry Management: A web form for prospective students to submit queries, which are directly inserted into the database.
- Admin Panel: A secure login portal for administrators to view a tabular list of inquiries, filter them by date or course, update their resolution status, and delete obsolete records.

Limitations and Exclusions:
- The system is strictly an inquiry and information dissemination platform. It does not process financial transactions, application fees, or tuition payments.
- It does not handle the core student information system (SIS) functionalities such as course registration, attendance tracking, grading, or alumni management.

- Third-party API integrations (such as SMS gateways or external AI LLMs for the chatbot) are currently outside the scope of this baseline implementation.

1.7 Significance of the Project

Deploying this system yields significant strategic advantages for the educational institution. Firstly, it acts as a digital brochure and the first point of interaction, establishing a highly professional and modern brand image. It empowers prospective students with self-service tools, greatly enhancing their initial experience with the college.

Operationally, the system transitions the college towards a paperless environment, aligning with eco-friendly initiatives. By leveraging Python and open-source technologies, the solution is highly cost-effective and extensible. In the long term, the structured data collected through this platform can be analyzed using data mining techniques to identify trends in student interests, evaluate the effectiveness of marketing campaigns, and forecast enrollment numbers for upcoming academic sessions.

1.8 Organization of the Dissertation

To present the research, methodology, and implementation in a structured manner, this dissertation is organized into seven distinct chapters:

Chapter 1 introduces the project's background, identifies the core problem, and outlines the objectives, scope, and overall significance.

Chapter 2 reviews the existing literature and similar systems, analyzing the technological landscape and highlighting the limitations of current manual processes.

Chapter 3 delves into the system analysis, detailing the functional and non-functional requirements, and specifying the hardware and software prerequisites for deployment.

Chapter 4 illustrates the system design and methodology, utilizing Data Flow Diagrams (DFDs), Entity-Relationship (ER) diagrams, and database schemas to map the architectural logic.

Chapter 5 provides an in-depth look at the implementation phase, including the development environment, major software modules, and critical code snippets that power the application.

Chapter 6 focuses on software testing, presenting the testing methodologies used and documenting various test cases and their results to validate the system's reliability.

Chapter 7 concludes the dissertation by summarizing the project's achievements, discussing its practical limitations, and suggesting avenues for future enhancements.

CHAPTER 2 - LITERATURE REVIEW / BACKGROUND STUDY

2.1 Introduction to the Subject Area

Admission enquiry is one of the first points of contact between a college and a prospective student. In a traditional system, students usually depend on the reception desk, printed brochures, phone calls, or direct meetings to get information about courses and admission. Such a process becomes difficult to manage when many students make enquiries at the same time.

Web applications provide a practical way to move this first stage of communication online. A student can open the college website from a desktop or mobile browser, read basic information, and submit an enquiry without depending on office hours. The proposed project focuses on this limited but useful part of the admission process. It is not a complete student information system and does not replace the full admission workflow.

The project uses a simple web stack. Python and Flask are used for server-side processing, Jinja2 is used to display dynamic HTML pages, JavaScript supports browser-side interactions such as the chatbot, and MySQL stores enquiry and administrator data. The combination is suitable for a small BCA project because the tools are widely documented and can be developed on a normal computer. The Flask and Jinja references used in the project are listed in the bibliography.

2.2 Existing Systems and Related Work

The existing admission enquiry methods seen in the project study can be grouped into four common types. Each type solves part of the problem, but each also has limitations for a small college that wants a simple system under its own control.

2.2.1 Traditional Manual System

In the manual approach, students visit the college or contact the reception desk. The enquiry is written on paper and is later entered into a register or spreadsheet. The main advantage is that staff can talk to the student directly. The problem is that the same information has to be collected again and again. Paper records are also difficult to search, update, and summarize.

2.2.2 Online Forms and Spreadsheets

Online forms reduce paper work and can collect data automatically. A simple form, however, is mainly a data collection tool. It does not always provide a college-specific interface, a separate admin area, or a clear workflow for reviewing individual enquiries. The proposed project adds these functions inside the same application.

2.2.3 Large Educational ERP Systems

Large ERP products can support many institutional activities, including admissions, student records, fees, and reporting. They are useful for large organizations but can be more difficult to configure for a small project. A small college enquiry application can be easier to understand and maintain when it contains only the modules needed for the enquiry process.

2.2.4 Third-Party Education Portals

Third-party education portals can help students discover colleges and can also forward leads. However, the college has less control over the design and the enquiry data is handled through an external service. The proposed system keeps the enquiry form and dashboard within the college application described in the report.

2.3 Review of Similar Applications and Technologies

The bibliography contains references for Flask, Python, MySQL Workbench, Jinja2, HTML/CSS standards, and software engineering. These sources support the technology choices used in the project. The project does not depend on a large external platform; instead, it combines a web framework, a database, a template engine, and browser scripting.

Flask is used because the application needs request routing, form handling, sessions, and template rendering without a very large framework. The project structure places the main server logic in app.py and keeps HTML templates under the templates directory. MySQL is used for the enquiry records and administrator data, while MySQL Workbench is used to create and inspect the database structure.

Jinja2 helps the application reuse common page elements such as navigation and footer sections. This can reduce repeated HTML and makes it easier to show dynamic database results on the admin dashboard. HTML, CSS, and JavaScript are used for the public

interface and client-side behaviour. The chatbot is an example of a small JavaScript feature that works without an external AI service.

2.4 Limitations of Existing Systems

Existing method	Main problem	Effect on enquiry work
Paper/register	Records are manual and harder to search	More time required for retrieval and follow-up
Basic online form	Mostly collects data without a management workflow	Staff still need another method to review records
Large ERP	May be more complex than the project needs	Higher setup and training effort
Third-party portal	College has less control over the application	Leads and presentation depend on the external service

2.5 How the Proposed System Differs

The proposed system focuses only on the admission enquiry stage. It provides public information pages, a digital enquiry form, a browser-based chatbot, and an admin area. This makes the project small enough to develop and demonstrate while still covering important BCA topics such as web routing, database access, form validation, sessions, HTML templates, JavaScript, and testing.

Another difference is that the project keeps the design and source code under the control of the college. The student interface and the admin interface are part of the same application. The system also stores enquiry records in MySQL so they can be retrieved later instead of remaining only in a paper file.

2.6 Summary of the Review

The review shows that the main value of the proposed project is not the use of a new technology but the way familiar technologies are combined for a focused problem. The system is simple, demonstrable, and suitable for a BCA project. Its limitations are also clear: it is an enquiry system, not a complete admission or student management platform.

CHAPTER 3 - SYSTEM ANALYSIS AND REQUIREMENTS

3.1 Existing System

The existing process described in the report is largely manual. Students may need to visit the college, ask for information, and fill in an enquiry form. Staff then enter or maintain the details using paper records or a local spreadsheet. When the number of enquiries grows, searching records and following up with students becomes difficult. The process also depends on staff availability during office hours.

3.2 Proposed System

The proposed system provides a web portal that students can use at any time. Public pages present college and course information. A student can complete the enquiry form from the browser and submit the data to the Flask backend. The backend sends the data to MySQL using a parameterized query. Administrators use a separate login page and dashboard to review submitted enquiries.

The screenshots in the existing dissertation also show a course-oriented public interface, an enquiry form with personal and academic information, an admin login screen, an admin dashboard, and a chatbot window. These screenshots are retained in this final report because they show the actual interface already included in the source dissertation.

3.3 Functional Requirements

Functional requirements describe what the application must do. The requirements below are based on the features already documented in the existing dissertation and its screenshots.

ID	Function	Requirement
FR-01	Public pages	Display home, about, courses, faculty, admissions and contact information.
FR-02	Enquiry form	Accept student enquiry details and send them to the backend.
FR-03	Form validation	Use browser validation for required fields and email format.
FR-04	Enquiry storage	Insert valid enquiry data into the MySQL Enquiries table.
FR-05	Chatbot	Show predefined responses for supported keywords or common questions.
FR-06	Admin login	Check administrator credentials and create a login session.
FR-07	Admin dashboard	Display submitted enquiry records in a tabular dashboard.
FR-08	View enquiry	Open an individual enquiry record from the dashboard.
FR-09	Logout	End the administrator session and return to the public/login area.
FR-10	Restricted admin routes	Prevent access to protected pages without a valid session.

Table 3.1: Functional Requirements

3.4 Non-Functional Requirements

Quality	Requirement
Security	Admin pages should be protected by session checks and database operations should use parameterized queries.
Usability	The public interface should be understandable to students with normal web browsing skills.
Responsiveness	The interface should work on desktop and mobile screen sizes.
Performance	The application should remain lightweight enough for the expected enquiry workload.
Reliability	Successful submissions should be stored in the database and failed transactions should be rolled back.
Maintainability	HTML views and Python logic should remain separated through Flask and Jinja templates.

Table 3.2: Non-Functional Requirements

3.5 User Requirements

The student user needs a clear way to find course and admission information, open the enquiry form, enter personal and academic details, and submit the request. The student also needs a quick way to ask common questions. The admin user needs a protected login, a dashboard showing enquiries, and access to individual records. These requirements are small enough to be handled by a single web application.

3.6 Hardware Requirements

Item	Minimum / Recommended Requirement
Processor	Intel Core i3 / AMD Ryzen 3 or equivalent
RAM	Minimum 4 GB; 8 GB recommended when database and server run together
Storage	At least 10 GB free space for tools, project files and database
Network	Broadband connection for package installation and future deployment
Browser	Modern web browser such as Chrome, Edge or Firefox

Table 3.3: Hardware Requirements

3.7 Software Requirements

Software / Tool	Use in Project
Operating system	Windows, Linux or another system that supports Python and MySQL
Programming language	Python 3.8 or higher
Web framework	Flask
Template engine	Jinja2
Database	MySQL Server 8.0+
Database tool	MySQL Workbench
Frontend	HTML5, CSS3 and JavaScript
Python packages	flask, pymysql, werkzeug.security, python-dotenv as described in the

	report
IDE	Visual Studio Code or PyCharm

Table 3.4: Software Requirements

3.8 Feasibility Study

A feasibility study helps check whether the project can be built and used with the available resources. The project is small, uses commonly available software, and does not require special hardware. The main areas considered are technical, economic, and operational feasibility.

Area	Observation	Result
Technical	Python, Flask and MySQL are suitable for the application and can be developed on a normal computer.	Feasible
Economic	The core development tools used in the report are free or open source, so software licensing cost is low for the prototype.	Feasible
Operational	The public pages and admin dashboard are simple web interfaces that can be learned by staff with basic computer skills.	Feasible

Table 3.5: Feasibility Summary

3.9 Requirement Summary

The analysis confirms that the project needs four main groups of functions: public information, enquiry submission, administrator access, and chatbot support. MySQL provides the persistent storage for the two documented tables. The next chapter explains how these parts are arranged and how data moves through the system.

3.10 Security and Data Integrity Requirements

The enquiry system handles personal contact information, so basic protection is needed even in a small prototype. The documented design uses sessions for the admin area, hashed administrator passwords, and parameterized SQL for database operations. Browser-side validation also reduces accidental incomplete submissions. These measures do not make the application secure against every possible attack, but they provide a reasonable baseline for the project scope.

Control	Project Requirement
Authentication	Only an authenticated admin session should open protected dashboard pages.
Password storage	Administrator passwords should be stored as hashes rather than plain text.
SQL safety	User values should be passed as query parameters rather than joined into SQL strings.
Transaction safety	Failed database operations should use rollback() before the connection is closed.
Input checking	Required fields and email format should be checked before submission.

Table 3.6: Security and Data Integrity Requirements

3.11 Analysis Summary

The analysis stage gives a clear boundary to the project. Students need a simple information and enquiry interface. Administrators need a protected view of the submitted data. The database needs only the fields required for the current enquiry workflow. With these boundaries, the design can remain small and understandable while still demonstrating important BCA concepts.

CHAPTER 4 - SYSTEM DESIGN AND METHODOLOGY

4.1 Development Methodology

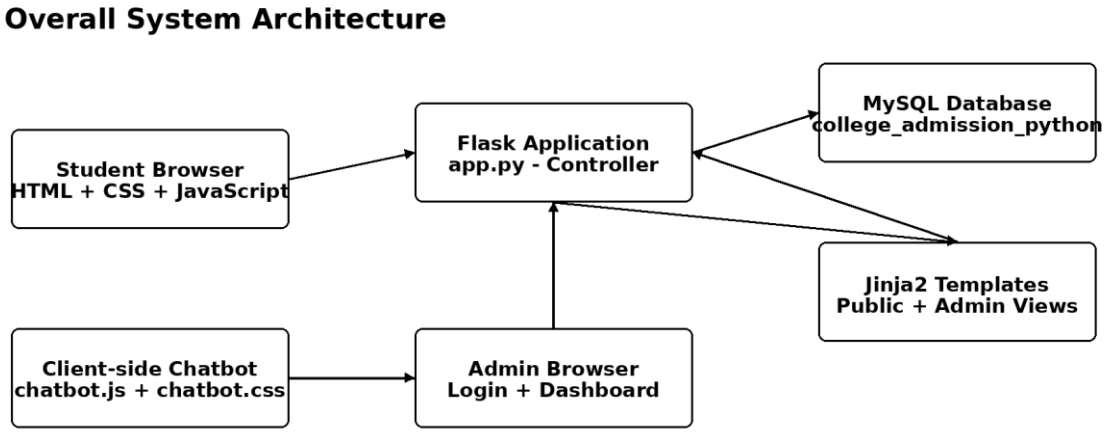
The existing report describes a modified Agile approach with small development phases. For this project, an iterative method is useful because pages, database fields, validation rules, and chatbot responses can be added and tested one part at a time. The phases used in the report are summarized below.

Phase	Work	Main Output
Phase 1	Requirements and database	Identify enquiry fields and prepare the MySQL tables.
Phase 2	Backend routing	Set up Flask, database connection, routes and session handling.
Phase 3	Frontend and Jinja	Create common layout, public pages, enquiry form and admin views.
Phase 4	Integration and chatbot	Connect form submission to the backend and add JavaScript chatbot logic.
Phase 5	Testing and security	Test validation, login, restricted routes, database operations and interface behaviour.

Table 4.1: Development Phases

4.2 System Architecture

The system uses a simple three-part arrangement. The browser provides the user interface, Flask handles request processing and session-based admin access, and MySQL stores the enquiry and administrator records. Jinja2 connects server-side data with the HTML templates. The chatbot runs in the browser and uses predefined JavaScript responses.



The figure reflects the architecture described in the report: Flask handles requests, MySQL stores data, Jinja renders pages, and the chatbot runs in the browser using predefined JavaScript responses.

Figure 4.1: Overall System Architecture

4.3 System Flow

For an enquiry, the student completes the browser form and the browser checks basic validation rules. The form data is sent to the Flask backend. The backend inserts the data into MySQL using a parameterized SQL statement. If the insertion succeeds, the application returns a success response; if there is a database error, the transaction is rolled back and an error response is returned.

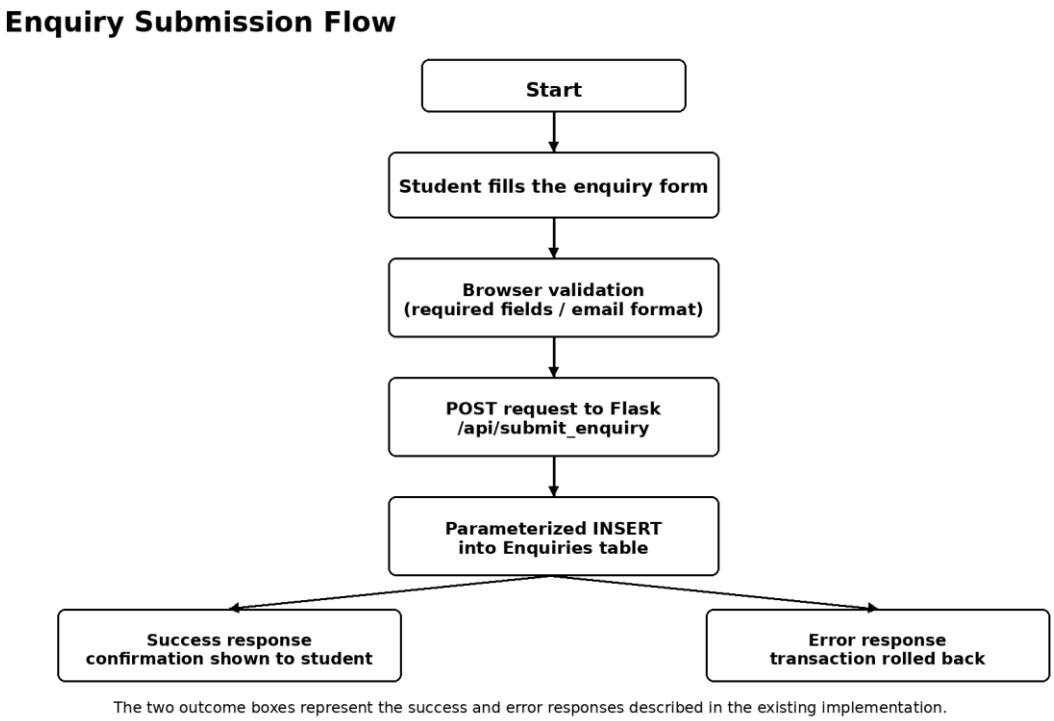


Figure 4.2: Enquiry Submission Flow

4.4 Database Design

The database named `college_admission_python` is described in the existing report. The main tables are `enquiries` and `admin`. The enquiries table stores student details, course interest, message text, and submission time. The admin table stores administrator account information and a password hash.

4.4.1 Enquiries Table

Field	Type	Constraint	Purpose
id	INT	Primary key, auto-increment	Unique enquiry identifier

name	VARCHAR(100)	NOT NULL	Student name stored by the enquiry form
email	VARCHAR(100)	NOT NULL	Student email address
phone	VARCHAR(20)	NOT NULL	Student contact number
course_interest	VARCHAR(50)	NOT NULL	Selected course
message	TEXT	NULL	Optional message or query
submitted_at	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP	Time when the record is stored

Table 4.2: Enquiries Table Data Dictionary

4.4.2 Admin Table

Field	Type	Constraint	Purpose
admin_id	INT	Primary key, auto-increment	Administrator identifier
username	VARCHAR(50)	NOT NULL, UNIQUE	Login username
password_hash	VARCHAR(255)	NOT NULL	Stored password hash

Table 4.3: Admin Table Data Dictionary

4.5 Database Relationships

The existing report treats the Admin and Enquiries tables as separate flat tables in the current implementation. At the application level, an administrator can manage multiple enquiries, but there is no documented foreign-key field from Enquiries to Admin in the current data dictionary. Therefore, the final report does not add a database relationship that is not already documented.

4.6 Security Design

The report describes session checks for protected admin routes. It also shows parameterized SQL in the enquiry insert statement. These are two important controls for a small application. The secret key is loaded from the environment when available, and the database password can also be supplied through environment variables. The admin password is described as a stored hash rather than plain text.

4.7 Design Summary

The design keeps the project easy to follow. A student mainly interacts with HTML pages and JavaScript. Flask receives requests and prepares responses. MySQL stores the data that must remain available between visits. The design also separates the public templates from the admin templates so that the two areas can be maintained independently.

4.8 Input and Output Design

The system receives input mainly from two user groups. Students provide enquiry details through the public form, while administrators provide login credentials and use dashboard controls. The output is shown as HTML pages, confirmation or error messages, enquiry records, and chatbot responses. Keeping the input and output clear is important because the project is intended for normal users rather than technical staff.

User / Source	Input	Main Output
Student	Personal and enquiry details	Confirmation message after successful submission
Student	Chatbot question	Predefined text response
Administrator	Username and password	Dashboard after successful login or an error on failure
Administrator	Request to view enquiry	Selected enquiry details
System	Database operation result	Success response or error response

Table 4.4: Input and Output Design

4.9 Route and Module Summary

The existing implementation described in the report uses separate routes for public pages, enquiry submission, administrator login, dashboard access, and individual enquiry views. The exact route list may grow as the application is extended, but the important idea is that each route has one clear responsibility. This keeps the controller easier to understand.

Route	Method	Purpose
/api/submit_enquiry	POST	Receives enquiry data and inserts a record into the Enquiries table.
/admin/dashboard	GET	Checks the session and displays saved enquiries.
/admin/login	GET/POST	Provides administrator authentication as described in the project structure.
/admin/view/<id>	GET	Displays the selected enquiry record in the admin area.
/admin/logout	GET	Ends the admin session as described by the dashboard workflow.

Table 4.5: Main Route and Module Summary

4.10 Validation and Error Handling Design

Validation is used at two levels in the documented project. The browser checks basic form rules such as required fields and email format before the request is sent. The Flask backend then receives the data and performs the database operation. When the database insert succeeds, the transaction is committed. When an exception occurs, the existing code calls rollback() and returns an error response. This two-step approach helps reduce incomplete input and protects the database transaction.

4.11 Design Summary

The final design is intentionally simple. It separates public and admin areas, keeps data in MySQL, uses Flask as the controller, and keeps the chatbot on the client side. The design also leaves room for future features without requiring the current prototype to become unnecessarily complicated.

4.12 User Roles and Permission Design

The current project has two main user roles. A student uses the public side of the website and does not need an account. An administrator uses the protected admin area. Separating the two roles reduces the chance that public users can directly open management pages.

Role	Main Access	Authentication
Student	Home, About, Courses, Faculty, Admissions, Contact, Enquiry, Chatbot	No login required
Administrator	Login, Dashboard, View Enquiry, Logout	Valid admin session required

Table 4.6: User Roles and Permission Design

4.13 Design Decisions

Several design choices were made to keep the project suitable for its academic scope. Flask was used instead of a larger framework, MySQL was selected for relational storage, Jinja2 was used for reusable HTML templates, and the chatbot was kept as browser-side JavaScript. These choices reduce setup complexity while still demonstrating routing, database connectivity, templating, authentication, and client-side scripting.

The system is also designed so that data moves in a clear direction. A student enters information in the browser, Flask receives the request, MySQL stores the enquiry, and the admin template later reads the stored record. This simple flow is useful during development because each part can be tested separately.

CHAPTER 5 - IMPLEMENTATION

5.1 Development Environment

The implementation described in the report uses Python with Flask, MySQL, Jinja2, HTML, CSS and JavaScript. MySQL Workbench is used for database management. The project can be developed in Visual Studio Code or PyCharm. The application is organized so that the main server logic is kept in app.py, templates are stored in the templates directory, and browser assets are stored under static.

5.2 Project Directory Structure

The existing report provides the following project structure. It is reproduced in a shorter format below so that the organization of the implementation is clear.

```
college_project-main/
  app.py
  requirements.txt
  database/database.sql
  static/
    css/style.css
    images/
    js/main.js
    js/enquiry.js
    chatbot/chatbot.css
    chatbot/chatbot.js
    admin/css/admin.css
  templates/
    base.html
    index.html
    about.html
    courses.html
    course_details.html
    faculty.html
    admissions.html
    enquiry.html
    contact.html
    404.html
    admin/base.html
    admin/login.html
    admin/dashboard.html
    admin/enquiries.html
    admin/view_enquiry.html
```

5.3 Main Modules of the System

Module	Implementation Role
Public information	Home, About, Courses, Faculty, Admissions and Contact pages provide college information.
Enquiry form	Collects student information and sends it to the Flask backend.
Chatbot	Uses JavaScript keyword matching and predefined answers.
Admin authentication	Provides a login page and session-based access control.
Admin dashboard	Displays stored enquiries and provides access to individual records.

Database layer	Stores enquiries and administrator credentials in MySQL.

Table 5.1: Main Project Modules

5.4 Backend Implementation

The Flask application in app.py is the main controller. It loads configuration values, creates the Flask application, connects to MySQL through PyMySQL, defines routes, and manages the administrator session. The report uses environment variables for the database host, port, username, password, database name, and secret key.

5.4.1 Database Connection

```
import os
from flask import Flask, render_template, request, redirect, url_for, flash,
session, jsonify
import pymysql
from dotenv import load_dotenv

load_dotenv()
app = Flask(__name__)
app.secret_key = os.getenv('SECRET_KEY', 'default_secret_key')

def get_db():
    return pymysql.connect(
        host=os.getenv('DB_HOST', '127.0.0.1'),
        port=int(os.getenv('DB_PORT', 3306)),
        user=os.getenv('DB_USER', 'root'),
        password=os.getenv('DB_PASS', ''),
        database=os.getenv('DB_NAME', 'college_admission_python'),
        charset='utf8mb4',
        cursorclass=pymysql.cursors.DictCursor
    )
```

This code creates the Flask application and a helper function named get_db(). The helper opens a connection using the configuration values. Keeping the connection code in one function makes the remaining routes easier to read.

5.4.2 Enquiry Submission

The enquiry submission route reads the form fields from request.form. The existing implementation uses a parameterized INSERT statement. The values are passed separately from the SQL command, so user input is treated as data rather than being joined directly into the SQL string.

```
@app.route('/api/submit_enquiry', methods=['POST'])
def submit_enquiry():
    name = request.form.get('name')
    email = request.form.get('email')
    phone = request.form.get('phone')
    course = request.form.get('course')
    message = request.form.get('message')

    conn = get_db()
    if conn:
        with conn.cursor() as cursor:
            sql_query = """
                INSERT INTO enquiries
                (name, email, phone, course_interest, message)
                VALUES (%s, %s, %s, %s, %s)
            """
            try:
                cursor.execute(sql_query, (name, email, phone, course,
message))
                conn.commit()
                return jsonify({"status": "success", "message": "Enquiry
submitted successfully."})
            except Exception as e:
                conn.rollback()
                return jsonify({"status": "error", "message": str(e)})
            finally:
                conn.close()
    return jsonify({"status": "error", "message": "Database error."})
```

The final report does not display the raw database exception to the user. A simple error message is better for a student-facing application and keeps internal database details out of the interface.

5.4.3 Admin Dashboard Access

The report checks the session before serving the dashboard. An unauthenticated request is redirected to the login page. For an authenticated request, the application reads the enquiry records from MySQL and passes them to the Jinja template.

```
@app.route('/admin/dashboard')
def admin_dashboard():
    if 'admin_id' not in session:
        flash('Unauthorized access. Please login first.', 'danger')
        return redirect(url_for('admin_login'))

    conn = get_db()
    if conn:
        with conn.cursor() as cursor:
            cursor.execute("SELECT * FROM enquiries ORDER BY submitted_at
DESC")
            all_enquiries = cursor.fetchall()
        conn.close()
        return render_template('admin/dashboard.html',
enquiries=all_enquiries)

    return 'Database Connection Failed', 500
```

5.5 Frontend Implementation

The frontend uses Jinja2 templates so that common elements such as the navigation and footer can be defined once. Individual templates extend the base layout and provide the page-specific content. The admin area has its own base template because its navigation and dashboard controls differ from the public site.

5.5.1 Admin Dashboard Template

```
{% extends 'admin/base.html' %}

{% block content %}
<table class='admin-table'>
  <thead>
    <tr>
      <th>ID</th><th>Student Name</th><th>Contact Info</th>
      <th>Course Interest</th><th>Date Submitted</th><th>Action</th>
    </tr>
  </thead>
  <tbody>
    {% for eq in enquiries %}
    <tr>
      <td>{{ eq.id }}</td>
      <td>{{ eq.name }}</td>
      <td>Email: {{ eq.email }}<br>Phone: {{ eq.phone }}</td>
      <td>{{ eq.course_interest }}</td>
      <td>{{ eq.submitted_at }}</td>
      <td><a href='/admin/view/{{ eq.id }}'>View Details</a></td>
    </tr>
    {% else %}
    <tr><td colspan='6'>No inquiries found in the database.</td></tr>
    {% endfor %}
  </tbody>
</table>
{% endblock %}
```

The loop receives the list named enquiries from the Flask route. For each record, the template prints the ID, name, contact information, course, date, and a link for more details. The else branch displays a message when there are no records.

5.6 Chatbot Integration

The chatbot runs on the client side. It listens for a user message, converts the text to lower case, checks it against a small dictionary of keywords, and displays the predefined answer. It does not call an external AI service. This makes the feature easy to demonstrate and keeps the chatbot within the scope of the project.

```
// static/chatbot/chatbot.js - Logic for Automated Replies
const chatInput = document.getElementById("chat-input");
const chatBody = document.getElementById("chat-body");

// Dictionary of predefined keywords and responses
const botResponses = {
    "hello": "Hi there! Welcome to B.J.S. Rampuria Jain College. How can I
help you today?",
    "courses": "We offer excellent programs in BCA, BBA, and B.Com. Visit
our Courses page for more details.",
    "fees": "Fee structures vary by program. BCA is typically ₹XX,XXX per
semester. Please submit an enquiry for exact figures.",
    "admission": "Admissions are currently open! Please fill out the Enquiry
Form to start the process.",
    "default": "I'm sorry, I didn't quite understand that. Please try asking
about 'courses', 'fees', or 'admission', or fill out our enquiry form for
human assistance."
};

chatInput.addEventListener("keypress", function(event) {
    if (event.key === "Enter") {
        let userText = chatInput.value.toLowerCase().trim();

        // Render User Message
        appendMessage(userText, "user-msg");
        chatInput.value = ""; // Clear input field

        // Determine Bot Response
        let botReply = botResponses["default"];
        for (let key in botResponses) {
            if (userText.includes(key)) {
                botReply = botResponses[key];
                break;
            }
        }

        // Simulate thinking delay then render Bot Message
        setTimeout(() => {
            appendMessage(botReply, "bot-msg");
        }, 500);
    }
});

function appendMessage(text, className) {
    let msgDiv = document.createElement("div");
    msgDiv.className = "message " + className;
    msgDiv.innerText = text;
    chatBody.appendChild(msgDiv);
    // Auto scroll to bottom
    chatBody.scrollTop = chatBody.scrollHeight;
}
```

The chatbot uses a small fixed set of keywords. The rule is simple: the message is converted to lower case and each supported keyword is checked. When a keyword is found, the matching predefined response is shown. When there is no match, the default response is used. This approach is easy to test and does not require an external API.

Keyword	Response Purpose
hello	Greeting and welcome message
courses	Information about BCA, BBA and B.Com
fees	Fee-related reply asking the user to submit an enquiry for exact figures
admission	Admission enquiry guidance

default	Fallback message for unsupported questions

Table 5.2: Chatbot Keyword Rules

5.8 Implementation Screenshots

The following screenshots are copied from the existing dissertation without changing their visual content. The captions are added in the required report format so that each screenshot has a clear figure number and description.

Home page showing the main navigation, hero section, course information and public chatbot icon.

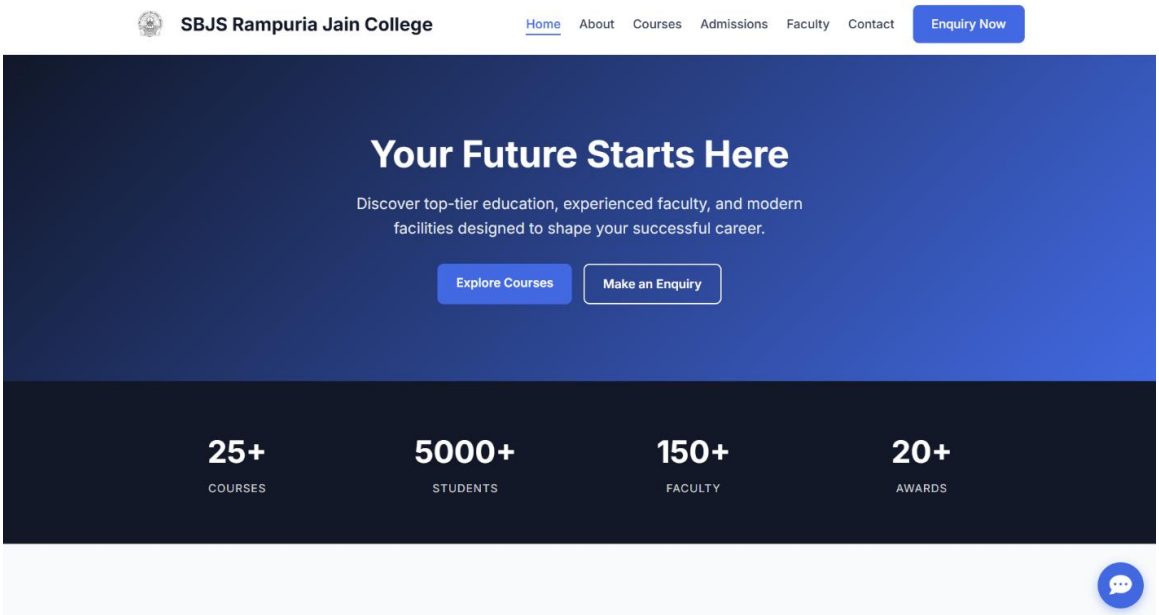


Figure 5.1: Home Page

Top part of the enquiry form showing personal information fields.

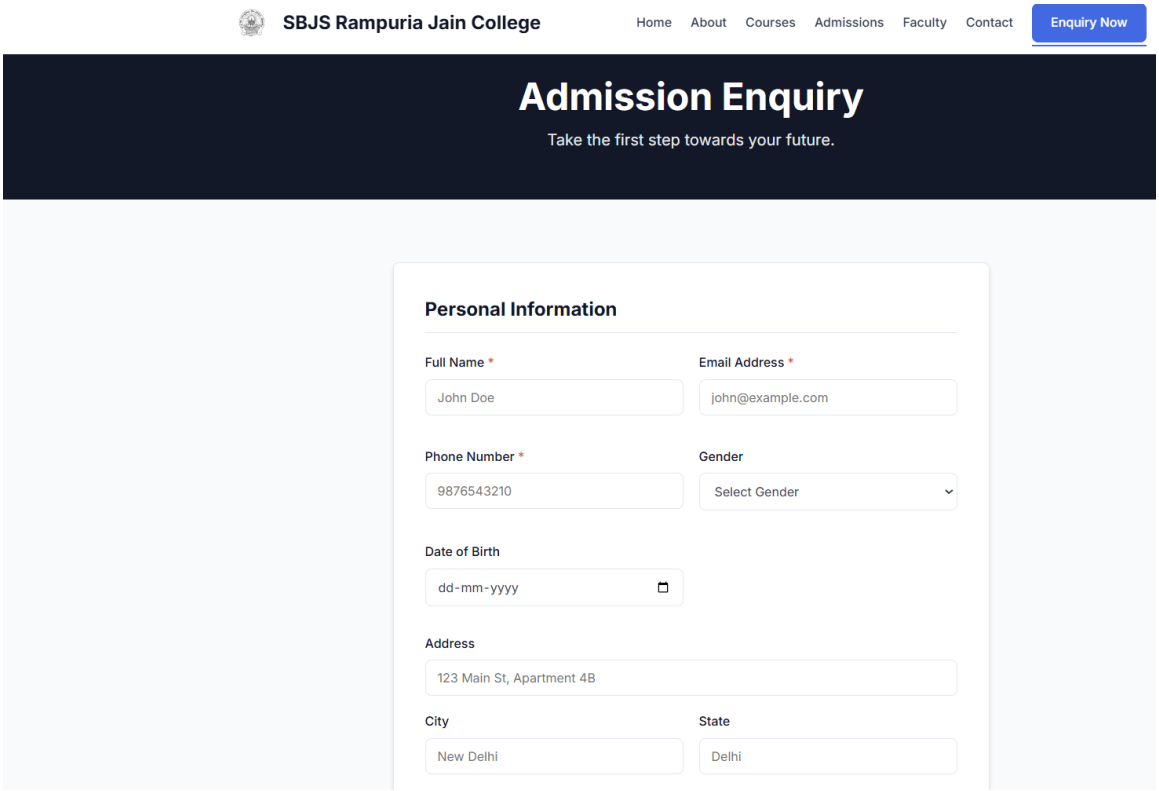


Figure 5.2: Admission Enquiry Form - Personal Information

Lower part of the enquiry form showing qualification, course, admission year and message fields.

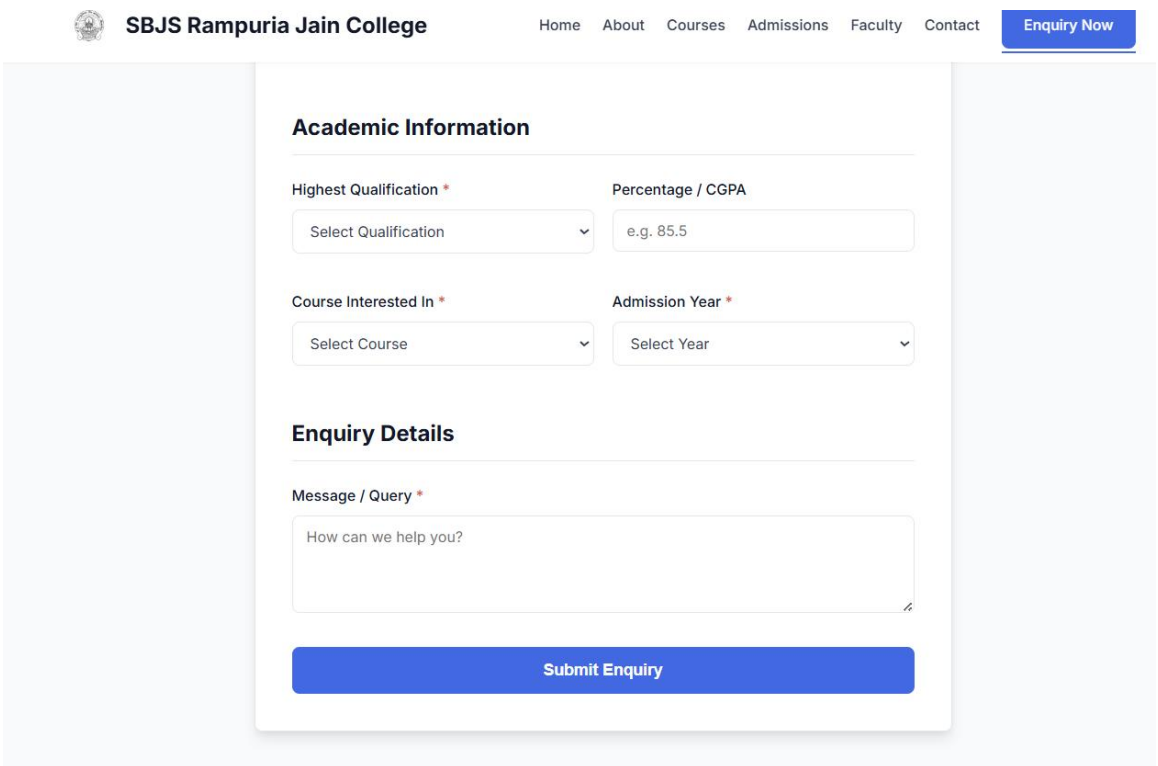


Figure 5.3: Admission Enquiry Form - Academic and Query Details

Login page used to enter administrator credentials.

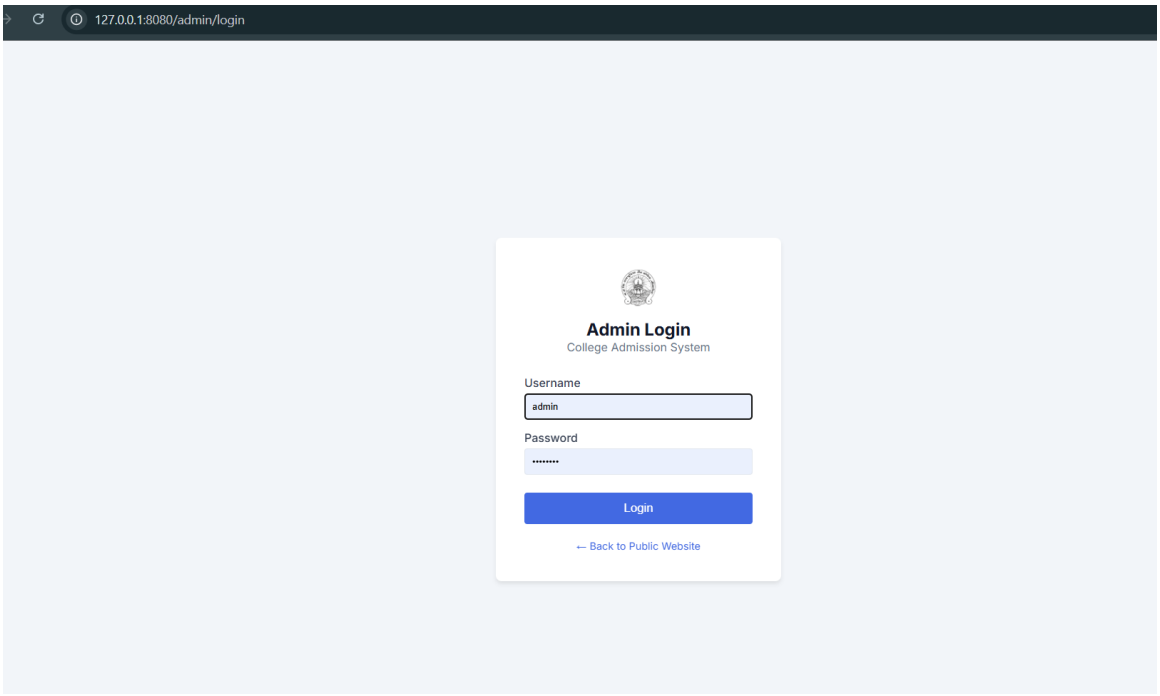


Figure 5.4: Admin Login Page

Dashboard showing enquiry records, summary cards and actions available to the administrator.

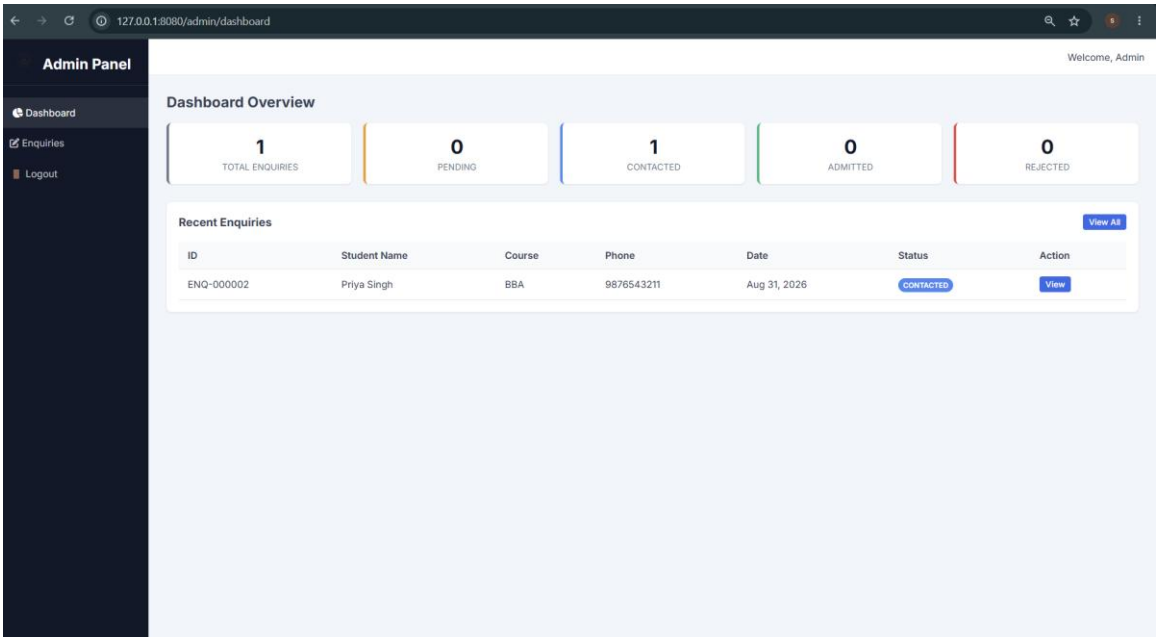


Figure 5.5: Admin Dashboard

Chatbot window displaying a predefined response to a user question.

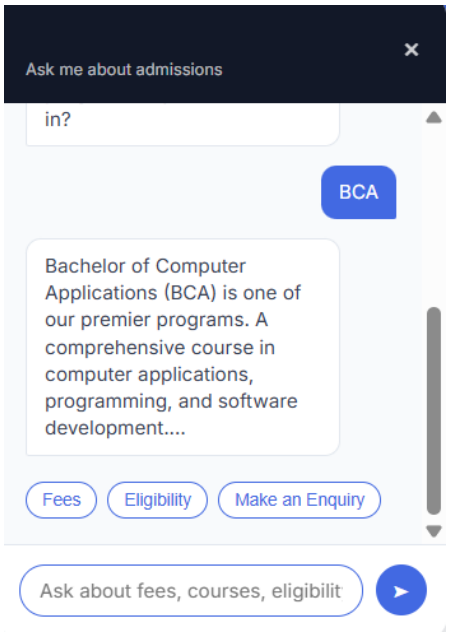


Figure 5.6: Chatbot Interaction

5.9 Implementation Summary

The implementation connects the public website, enquiry form, Flask backend, MySQL database, admin dashboard and client-side chatbot into one project. The code shown in this chapter contains only the important parts needed to explain the working of the application. The full source code remains outside the report, as recommended by the dissertation guidelines.

5.10 Frontend Page Summary

The public side of the application is divided into separate templates so that each page has a clear purpose. The main navigation links the pages together. The enquiry page is the main data-entry page, while the admin templates are kept in a separate folder for staff-only functions.

Template	Page	Purpose
index.html	Home page	Main public entry point and navigation
about.html	About page	Institutional information
courses.html	Courses page	Program information
course_details.html	Course detail	Detailed course information

faculty.html	Faculty page	Faculty information
admissions.html	Admissions page	Admission guidance
enquiry.html	Enquiry page	Student enquiry form
contact.html	Contact page	Contact and location information
admin/login.html	Admin login	Administrator authentication
admin/dashboard.html	Dashboard	List and summary of enquiries
admin/view_enquiry.html	View enquiry	Details of one enquiry

Table 5.3: Frontend Page Summary

5.11 Database Operations and Error Handling

The database layer is used whenever an enquiry is created or when an administrator needs to read stored enquiries. The report shows a helper function for opening a connection and uses a cursor for SQL operations. After a successful insert, commit() makes the transaction permanent. If an exception occurs, rollback() is called. Finally, the connection is closed. This pattern is useful because it prevents an unfinished database transaction from being left open.

5.12 Administrative Workflow

The administrator workflow begins at the login page. After successful authentication, the user reaches the dashboard. The dashboard displays the enquiries fetched from MySQL. The administrator can open a particular record through the View Details link and can leave the protected area by using logout. The dashboard screenshot in Figure 5.5 also shows summary information and enquiry status values in the current interface.

5.13 Implementation Notes

The implementation is designed as a student-level prototype. The use of separate templates, static files, and a database keeps the code organized. The project does not require a large framework or an external chatbot service. This makes the system easier to run, explain, and test during the BCA viva.

5.14 Maintenance and Deployment Notes

For local development, the application requires Python, the project dependencies, and a running MySQL server. The database script in the project can be used to prepare the required tables. Configuration values can be kept in environment variables so that database credentials and the Flask secret key are not hard-coded into the application settings.

Before deployment, the administrator should create a proper password, review database permissions, configure a strong secret key, and test all public and protected routes again. The prototype can later be placed on a cloud or institutional server, but such deployment would require additional configuration and security checks that are outside the current project scope.

5.15 Implementation Summary

The implementation chapter shows the main technical parts of the system without printing the complete source code. The combination of short code snippets, project structure, module descriptions and genuine application screenshots gives a clear view of how the system is built and how the different components work together.

CHAPTER 6 - TESTING AND RESULTS

6.1 Testing Methodology

Testing was carried out to check whether the main modules work as expected. The project uses practical functional tests rather than complex automated testing. The tests cover public navigation, form validation, enquiry storage, administrator authentication, restricted dashboard access, dashboard data retrieval, view details, logout, and chatbot responses.

6.2 Test Cases

ID	Module	Test / Input	Expected Result	Actual Result	Status
TC-01	Public navigation	Open Courses page	Course page loads	Course page loaded	Pass
TC-02	Public navigation	Open About page	About page loads	About page loaded	Pass
TC-03	Enquiry form	Enter valid form data and submit	Success response and record saved	Confirmation shown and row added	Pass
TC-04	Enquiry validation	Leave required field empty	Browser stops submission	Validation message shown	Pass
TC-05	Enquiry validation	Enter invalid email	Browser rejects invalid email	Email validation message shown	Pass
TC-06	Database security	Enter SQL-like text in form	Input stored as data, no SQL manipulation	Literal text treated as input	Pass
TC-07	Admin login	Enter valid credentials	User is redirected to dashboard	Dashboard opened	Pass
TC-08	Admin login	Enter invalid credentials	Login rejected	Error message shown	Pass
TC-09	Admin security	Open dashboard without login	Redirect to login	Login page displayed	Pass
TC-10	Dashboard	Open dashboard with records	Records appear in table	Records displayed	Pass
TC-11	View enquiry	Click View Details	Selected record opens	Selected enquiry opened	Pass
TC-12	Logout	Click logout	Session ends	User logged out	Pass
TC-13	Chatbot	Type hello	Greeting shown	Greeting shown	Pass
TC-14	Chatbot	Type fees	Fee-related response shown	Fee-related response shown	Pass
TC-15	Chatbot	Type unknown message	Default response shown	Default response shown	Pass

Table 6.1: Detailed Test Cases

6.3 Test Results

The test cases documented in the existing report show successful results for the main functions. Public pages loaded correctly, the enquiry form accepted valid data, browser validation handled incomplete and invalid input, and the MySQL insert completed successfully for valid submissions. The admin login accepted valid credentials and rejected invalid credentials. Direct access to the dashboard without a session was redirected to the login page. The chatbot returned the expected predefined replies for supported keywords and a fallback response for unrecognized input.

6.4 Security Testing

Two security checks are especially important in this project. First, the admin dashboard checks the session before showing protected data. Second, the enquiry route uses parameterized SQL. The test case for SQL-like input confirms that the value is treated as a normal string instead of changing the SQL command.

6.5 Discussion of Results

The results indicate that the application meets the main functional goals described in the report. The system can display information, accept enquiries, store them in MySQL, and allow an administrator to review them. The chatbot provides quick answers for common questions. The testing also confirms that invalid form input and unauthenticated access are handled in a controlled manner.

The results should be understood within the scope of a prototype. The tests were performed on the features described in the project report and screenshots. They do not represent large-scale production load testing, formal penetration testing, or long-term operational monitoring. Such testing can be added in a future version when the application is deployed more widely.

6.6 Requirement-to-Test Traceability

The test cases can also be viewed as a simple requirement-to-test mapping. This makes it easier to see that the major functions listed in Chapter 3 were tested at least once.

Requirement	Function	Related Test Cases	Result
FR-01	Public pages	TC-01, TC-02	Pass
FR-02	Enquiry form	TC-03	Pass
FR-03	Form validation	TC-04, TC-05	Pass
FR-04	Enquiry storage	TC-03, TC-06	Pass
FR-05	Chatbot	TC-13, TC-14, TC-15	Pass
FR-06	Admin login	TC-07, TC-08	Pass
FR-07	Dashboard	TC-10	Pass
FR-08	View enquiry	TC-11	Pass
FR-09	Logout	TC-12	Pass
FR-10	Restricted admin routes	TC-09	Pass

Table 6.2: Requirement-to-Test Traceability

This mapping does not replace the detailed test table. It provides a quick check that the requirements and the recorded tests are connected.

6.7 Overall Test Summary

For a small academic project, a simple test summary is useful for showing the overall outcome without claiming large-scale performance results. The tests recorded in this report covered the main public functions, form behaviour, database insertion, admin access and chatbot rules.

Area	Tests	Passed	Overall
Public navigation	2	2	Pass
Enquiry form and validation	4	4	Pass
Security and admin access	4	4	Pass
Dashboard data and view	2	2	Pass
Chatbot responses	3	3	Pass
Total	15	15	Pass

Table 6.3: Overall Test Summary

6.8 Limitations of Testing

The testing performed for this dissertation focuses on functional behaviour that can be demonstrated on the development setup. It does not measure behaviour under a large number of simultaneous users, long-term uptime, or production network conditions. It also does not replace a formal security audit. These limits are consistent with the scope of a small BCA prototype and should be considered before production use.

CHAPTER 7 - CONCLUSION AND FUTURE SCOPE

7.1 Conclusion

The Web Based Admission Enquiry System was developed to provide a simple online method for handling admission enquiries. The project brings together a public information website, an online enquiry form, a MySQL database, an admin login and dashboard, and a small browser-based chatbot. These parts work together to reduce the need for paper enquiry forms and repeated basic questions at the college reception.

The project achieved the main objectives stated in Chapter 1. It uses Python and Flask for the backend, Jinja2 and HTML/CSS/JavaScript for the frontend, and MySQL for data storage. Form validation, parameterized SQL queries, and session checks were included as practical measures for data quality and access control. The project also provided practical experience in building and testing a complete small web application.

7.2 Major Findings

- A web-based enquiry form makes it easier for students to submit their questions without visiting the college for every basic enquiry.
- A database gives the administrator a central place to review stored enquiries.
- A simple chatbot can answer repeated questions without needing a third-party AI service.
- Session checks and parameterized SQL queries are useful security measures for a small Flask application.

7.3 Limitations

The system is limited to enquiry and information handling. It does not process application fees or tuition payments and it is not a complete student information system. The chatbot is rule-based, so it cannot understand every form of natural language question. The project also does not document an integrated email or SMS notification service. The current database design is intentionally simple and does not include a complete counselling workflow or advanced reporting system.

7.4 Future Scope

The future scope can focus on useful additions without changing the basic structure of the project. Email and SMS notifications can be added when an enquiry is submitted. The admin dashboard can be extended with search, filters, status tracking, and simple charts. A more advanced chatbot can be introduced later if the college needs natural language support. The application could also be connected to a payment service and document upload module when the project grows from enquiry management toward a wider admission workflow.

7.5 Final Summary

The project demonstrates that a small college admission enquiry process can be supported with familiar web technologies. The main outcome is a working prototype that is easy to demonstrate and explain during a viva. Its design is simple enough for the current project scope, while the structure provides a base for future improvements.

REFERENCES / BIBLIOGRAPHY

1. Grinberg, M. (2018). Flask Web Development: Developing Web Applications with Python. 2nd ed. O'Reilly Media.

2. Pallets Projects. (2026). Flask Documentation. Available at: https://flask.palletsprojects.com/

3. Oracle Corporation. (2026). MySQL Workbench User's Guide. Available at: https://dev.mysql.com/doc/workbench/en/

4. Jinja Documentation. (2026). Jinja2 Template Engine Official Documentation. Available at: https://jinja.palletsprojects.com/

5. Pressman, R. S. (2014). Software Engineering: A Practitioner's Approach. 8th ed. McGraw-Hill Education.

6. Python Software Foundation. (2026). Python 3 Language Reference. Available at: https://docs.python.org/3/

7. W3C (World Wide Web Consortium). (2026). HTML5 and CSS3 Specifications. Available at: https://www.w3.org/